RETAILMART V3 · SQL

JSON & Semi-Structured Data

— Siraj Sir

📱 WhatsApp Announcement

JSON & Semi-Structured Data

More and more data arrives as JSON -- API logs, event payloads, configuration blobs. PostgreSQL has first-class JSONB support that lets you query nested structures without an ETL pipeline.

Part 1: JSONB Operators (`->` `->>` `@>` `?`)
Part 2: Unnesting Arrays with jsonb_array_elements
Part 3: Flattening JSON into Typed Columns

After today, you can crack open any JSON column and pull structured data from it for analysis.

TODAY'S AGENDA (2 Hours)

Section	Time	Topics
Part 1: JSONB Operators	2 Hours	-> vs ->> (object vs text), #> and #>> for nested paths, Containment (@>) and existence (?) operators
Part 2: jsonb_array_elements		Unnesting JSON arrays into rows, Lateral joins with JSON functions, Aggregating back from unnested data
Part 3: Flattening JSON		Extracting and casting to typed columns, Building analysis-ready views from JSON, Lab: Parse audit.api_requests payloads

YOUR SQL JOURNEY

- ✓ LAG & LEAD
- ✓ Query Plans
- ✓ Indexing
- ✓ Pivoting & FILTER
- ✓ Median & DISTINCT ON
- ✓ Date/Time Mastery
- 24 **JSON & JSONB** ← HERE
- 25 Views & MVs

RECAP

What We Covered Yesterday

- generate_series for date dimensions and backbones
- Gap-filling with LEFT JOIN + COALESCE for complete time series
- Time bucketing with DATE_TRUNC and EXTRACT
- TIMESTAMPTZ and AT TIME ZONE for timezone handling

STORY

The API Log Column

In RetailMart, audit.api_requests stores its fields as typed columns (method, endpoint, status_code, response_time_ms, user_agent). In the wild, that same data often arrives as one nested JSONB 'payload'. To practice JSONB operators on realistic data, we reconstruct that payload from the columns with jsonb_build_object(...) AS payload -- exactly the shape an ingested API log would have -- then extract from it. (The one truly-JSONB column in RetailMart is audit.procedure_calls.input_params.)

DEFINITION

Always Use JSONB

- JSON: stores as text, re-parses on every access. Preserves whitespace.
- JSONB: stores in binary format, fast access, supports indexing.
- In practice, always use JSONB. JSON is only useful for exact text preservation.

JSONB Operators

```
-- -> (get JSON object)

 col -> 'key'       -- returns JSONB
 col -> 0           -- array index 0

-- ->> (get as text)

 col ->> 'key'      -- returns TEXT
 col ->> 0          -- array index as text

-- #> #>> (nested path)

 col #> '{a,b}'     -- nested, returns JSONB
 col #>> '{a,b}'    -- nested, returns TEXT
```

JSONB Operators

-- @> (contains)

```
col @> '{"status":200}'
  -- true if col contains this JSON
```

Arrow Operators

```
-- Given: {"method": "GET", "status": 200, "latency": 45.2}

-- -> returns JSONB (for chaining or further JSON ops):
payload -> 'method'          -- returns: "GET" (with quotes)

-- ->> returns TEXT (for display, comparison, casting):
payload ->> 'method'         -- returns: GET (no quotes)

-- For numbers, ->> gives text, so cast explicitly:
(payload ->> 'status')::INT  -- returns: 200 (as integer)
(payload ->> 'latency')::NUMERIC  -- returns: 45.2
```

Nested Path

```
-- Given: {"request": {"headers": {"user_agent": "Chrome"}}}

-- Step by step with ->:
payload -> 'request' -> 'headers' ->> 'user_agent'

-- Or use #>> with a path array:
payload #>> '{request,headers,user_agent}'

-- Both return: Chrome
```

Containment Operators

```
-- (api = api_requests with a JSON payload built from its columns)
WITH api AS (
  SELECT *, jsonb_build_object(
    'method', method, 'status_code', status_code,
    'latency_ms', response_time_ms
  ) AS payload
  FROM audit.api_requests
)
-- @> : does the JSONB contain this sub-document?
SELECT * FROM api
WHERE payload @> '{"method": "POST"}';

-- ? : does the JSONB have this key?  (use the api CTE again)
-- SELECT * FROM api WHERE payload ? 'error_message';

-- ?| : has ANY of these keys?
-- SELECT * FROM api
-- WHERE payload ?| ARRAY['error', 'error_message'];
```

PRO TIP

PRO TIP

The @> containment operator uses GIN indexes efficiently. If you frequently filter by a JSON field value, a GIN index on the JSONB column makes @> queries fast.

Practice 1: Extract Fields from JSONB

EASY

SCENARIO: The API monitoring team needs error rates by HTTP method.

TASK: From audit.api_requests, extract the method and status_code from the payload column. Count requests per method per status_code. Show the top 10 by count.

HINT: Use payload ->> 'method' for text, (payload ->> 'status_code')::INT for numbers.

Answer

✅ ANSWER

```
SELECT
  payload ->> 'method'           AS method,
  (payload ->> 'status_code')::INT
                                 AS status_code,
  COUNT(*)                        AS request_count
FROM (
  SELECT *, timestamp AS request_timestamp,
    jsonb_build_object(
      'method', method, 'endpoint', endpoint,
      'status_code', status_code,
      'latency_ms', response_time_ms,
      'user_agent', user_agent
    ) AS payload
  FROM audit.api_requests
) api
GROUP BY
  payload ->> 'method',
  (payload ->> 'status_code')::INT
ORDER BY request_count DESC
LIMIT 10;
```

Practice 2: Filter by JSONB Containment

MEDIUM

SCENARIO: Find all failed API requests (status_code >= 400) and calculate average latency for errors vs successes.

TASK: Compare average latency_ms for successful (status < 400) vs failed (status >= 400) requests. Use ->> with casting.

HINT: Cast both status_code and latency_ms from ->> text to numeric types.

Answer (1/2)

✅ ANSWER

```
SELECT
  CASE
    WHEN (payload ->> 'status_code')::INT >= 400
      THEN 'error'
    ELSE 'success'
  END                              AS category,
  COUNT(*)                          AS requests,
  ROUND(AVG(
    (payload ->> 'latency_ms')::NUMERIC
  ), 2)                            AS avg_latency_ms
FROM (
  SELECT *, timestamp AS request_timestamp,
    jsonb_build_object(
      'method', method, 'endpoint', endpoint,
      'status_code', status_code,
      'latency_ms', response_time_ms,
      'user_agent', user_agent
    ) AS payload
  FROM audit.api_requests
) api
WHERE payload ? 'status_code'
  AND payload ? 'latency_ms'
```

Answer (2/2)

✅ ANSWER

```
GROUP BY category;
```

STORY

The Tags Array

Some JSON columns contain arrays: a product has tags like ["electronics", "premium", "sale"]. To analyze by tag, you need to explode each array into separate rows. jsonb_array_elements does exactly this -- one row per array element.

Unnesting Arrays

```
-- Given JSON: {"tags": ["a", "b", "c"]}

-- Unnest into rows:
SELECT
  id,
  tag.value AS tag
FROM my_table,
  jsonb_array_elements(payload -> 'tags') AS tag(value);

-- Returns:
-- id | tag
-- 1  | "a"
-- 1  | "b"
-- 1  | "c"

-- Use ->> 0 or ::TEXT to strip quotes
-- or jsonb_array_elements_text() for text directly.
```

Text Version

```
-- Returns text instead of JSONB:
SELECT
  id,
  tag AS tag_text
FROM my_table,
  jsonb_array_elements_text(
    payload -> 'tags'
  ) AS tag;

-- Returns text without quotes:
-- id | tag_text
-- 1  | a
-- 1  | b
-- 1  | c
```

PRO TIP

PRO TIP

jsonb_array_elements is a set-returning function. Use it in a LATERAL join or implicit cross join in the FROM clause. It creates one row per array element, so your row count multiplies by the array length.

Practice 3: Unnest and Aggregate JSON Arrays

HARD

SCENARIO: API request payloads contain a 'headers' array. You need to find the most common user agents.

TASK: Extract the user_agent field from audit.api_requests payloads. Group by user_agent and count occurrences. Show the top 5 user agents.

HINT: If user_agent is a top-level field, use ->>. If it is nested inside headers, use #>> or chained ->/->> operators.

Answer

✅ ANSWER

```
SELECT
  payload ->> 'user_agent'       AS user_agent,
  COUNT(*)                        AS request_count
FROM (
  SELECT *, timestamp AS request_timestamp,
    jsonb_build_object(
      'method', method, 'endpoint', endpoint,
      'status_code', status_code,
      'latency_ms', response_time_ms,
      'user_agent', user_agent
    ) AS payload
  FROM audit.api_requests
) api
WHERE payload ? 'user_agent'
GROUP BY payload ->> 'user_agent'
ORDER BY request_count DESC
LIMIT 5;
```

STORY

The Analytics View

Raw JSONB columns are hard to work with in dashboards. The solution: create a view that extracts and casts JSON fields into proper typed columns. Analysts query the view like a normal table, never touching JSON syntax.

Flattening View (1/2)

```
-- Create an analysis-ready view from JSONB:
CREATE OR REPLACE VIEW analytics.v_api_requests AS
SELECT
  request_id,
  request_timestamp,
  payload ->> 'method'              AS method,
  payload ->> 'endpoint'            AS endpoint,
  (payload ->> 'status_code')::INT  AS status_code,
  (payload ->> 'latency_ms')::NUMERIC
                                    AS latency_ms,
  payload ->> 'user_agent'          AS user_agent,
  CASE
    WHEN (payload ->> 'status_code')::INT >= 400
      THEN true
    ELSE false
  END                               AS is_error
FROM (
  SELECT *, timestamp AS request_timestamp,
    jsonb_build_object(
      'method', method, 'endpoint', endpoint,
```

Flattening View (2/2)

```
      'status_code', status_code,
      'latency_ms', response_time_ms,
      'user_agent', user_agent
    ) AS payload
  FROM audit.api_requests
) api;

-- Now analysts can query:
-- SELECT method, AVG(latency_ms)
-- FROM analytics.v_api_requests
-- WHERE is_error = false
-- GROUP BY method;
```

Type Casting

```
-- Always cast ->> results to the right type:
(payload ->> 'count')::INT         -- integer
(payload ->> 'price')::NUMERIC     -- decimal
(payload ->> 'active')::BOOLEAN    -- boolean
(payload ->> 'created')::TIMESTAMP -- timestamp

-- Protect against missing keys:
COALESCE(
  (payload ->> 'latency_ms')::NUMERIC,
  0
)                                  -- default to 0
```

COMMON MISTAKE

Common JSON Mistakes

Using -> when you need ->>: -> returns JSONB (with quotes). ->> returns TEXT (no quotes). For display and comparison, use ->>. For chaining deeper, use ->.

Forgetting to cast ->> results: ->> always returns TEXT. If you compare to a number without casting, you get string comparison ('9' > '10' is true!).

Not handling missing keys: If a key does not exist, -> and ->> return NULL. Use COALESCE or WHERE payload ? 'key' to handle missing data.

Practice 4: API Error Rate by Endpoint

HARD

SCENARIO: The engineering team needs error rates per API endpoint.

TASK: From audit.api_requests, extract endpoint and status_code. Calculate total requests, error count (status >= 400), and error rate percentage per endpoint. Show the top 10 endpoints by error rate.

Answer (1/2)

✅ ANSWER

```
SELECT
  payload ->> 'endpoint'          AS endpoint,
  COUNT(*)                         AS total,
  COUNT(*) FILTER (WHERE
    (payload ->> 'status_code')::INT >= 400)
                                   AS errors,
  ROUND(
    COUNT(*) FILTER (WHERE
      (payload ->> 'status_code')::INT >= 400)
    * 100.0 / NULLIF(COUNT(*), 0), 2
  )                                AS error_rate_pct
FROM (
  SELECT *, timestamp AS request_timestamp,
    jsonb_build_object(
      'method', method, 'endpoint', endpoint,
      'status_code', status_code,
      'latency_ms', response_time_ms,
      'user_agent', user_agent
    ) AS payload
  FROM audit.api_requests
) api
WHERE payload ? 'endpoint'
```

Answer (2/2)

✅ ANSWER

```
  AND payload ? 'status_code'
GROUP BY payload ->> 'endpoint'
ORDER BY error_rate_pct DESC
LIMIT 10;
```

Practice 5: Full API Analytics Dashboard Query

CRAZY

SCENARIO: Build a comprehensive API analytics query that combines JSONB extraction with time-series and aggregation patterns.

TASK: Show hourly API metrics: total requests, error rate, median latency, P95 latency. Only for the last 24 hours. Group by hour.

Answer (1/2)

✅ ANSWER

```
SELECT
  DATE_TRUNC('hour', request_timestamp)
    AS hour,
  COUNT(*)                          AS requests,
  ROUND(
    COUNT(*) FILTER (WHERE
      (payload ->> 'status_code')::INT >= 400)
    * 100.0 / NULLIF(COUNT(*), 0), 1
  )                                 AS error_rate,
  ROUND(PERCENTILE_CONT(0.5)
    WITHIN GROUP (ORDER BY
      (payload ->> 'latency_ms')::NUMERIC
    )::NUMERIC, 1)                  AS median_latency,
  ROUND(PERCENTILE_CONT(0.95)
    WITHIN GROUP (ORDER BY
      (payload ->> 'latency_ms')::NUMERIC
    )::NUMERIC, 1)                  AS p95_latency
FROM (
  SELECT *, timestamp AS request_timestamp,
    jsonb_build_object(
      'method', method, 'endpoint', endpoint,
      'status_code', status_code,
```

Answer (2/2)

✅ ANSWER

```
      'latency_ms', response_time_ms,
      'user_agent', user_agent
    ) AS payload
  FROM audit.api_requests
) api
WHERE request_timestamp >= NOW() - INTERVAL '24 hours'
  AND payload ? 'status_code'
  AND payload ? 'latency_ms'
GROUP BY DATE_TRUNC('hour', request_timestamp)
ORDER BY hour;
```

INTERVIEW

Q: What is the difference between -> and ->> in PostgreSQL JSONB?

A: -> returns the value as JSONB type (preserving the JSON structure, with quotes on strings). ->> returns the value as TEXT. Use -> when you need to chain operators (payload -> 'a' -> 'b') or work with JSON values. Use ->> when you need the value for display, comparison, or casting to another type.

INTERVIEW

Q: How do you query inside a JSON array in PostgreSQL?

A: Use jsonb_array_elements(col -> 'array_key') to explode each array element into a row. For text arrays, use jsonb_array_elements_text(). You can then GROUP BY, filter, or aggregate the expanded rows. This is similar to UNNEST for PostgreSQL arrays.

HW 1: Extract Top-Level Fields

EASY

TASK: From audit.api_requests, extract method, endpoint, status_code, and latency_ms as typed columns. Limit to 20 rows.

Answer

✅ ANSWER

```
SELECT
  payload ->> 'method'            AS method,
  payload ->> 'endpoint'          AS endpoint,
  (payload ->> 'status_code')::INT
                                  AS status_code,
  (payload ->> 'latency_ms')::NUMERIC
                                  AS latency_ms
FROM (
  SELECT *, timestamp AS request_timestamp,
    jsonb_build_object(
      'method', method, 'endpoint', endpoint,
      'status_code', status_code,
      'latency_ms', response_time_ms,
      'user_agent', user_agent
    ) AS payload
  FROM audit.api_requests
) api
LIMIT 20;
```

HW 2: JSONB Containment Filter

MEDIUM

TASK: Find all POST requests with status_code 500 using the @> containment operator. Count them by endpoint.

Answer

✅ ANSWER

```
SELECT
  payload ->> 'endpoint' AS endpoint,
  COUNT(*)               AS error_count
FROM (
  SELECT *, timestamp AS request_timestamp,
    jsonb_build_object(
      'method', method, 'endpoint', endpoint,
      'status_code', status_code,
      'latency_ms', response_time_ms,
      'user_agent', user_agent
    ) AS payload
  FROM audit.api_requests
) api
WHERE payload @> '{"method": "POST",
  "status_code": 500}'
GROUP BY payload ->> 'endpoint'
ORDER BY error_count DESC;
```

HW 3: JSONB Key Existence Check

MEDIUM

TASK: Count how many API requests have an 'error_message' key in their payload vs how many do not. Show as two rows.

Answer

✅ ANSWER

```
SELECT
  CASE WHEN payload ? 'error_message'
    THEN 'has_error_message'
    ELSE 'no_error_message'
  END AS category,
  COUNT(*) AS request_count
FROM (
  SELECT *, timestamp AS request_timestamp,
    jsonb_build_object(
      'method', method, 'endpoint', endpoint,
      'status_code', status_code,
      'latency_ms', response_time_ms,
      'user_agent', user_agent
    ) AS payload
  FROM audit.api_requests
) api
GROUP BY (payload ? 'error_message')
ORDER BY category;
```

HW 4: Latency Percentiles by Method

HARD

TASK: Calculate median and P95 latency per HTTP method, using PERCENTILE_CONT on the extracted latency_ms field.

Answer (1/2)

✅ ANSWER

```
SELECT
  payload ->> 'method' AS method,
  COUNT(*) AS requests,
  ROUND(PERCENTILE_CONT(0.5)
    WITHIN GROUP (ORDER BY
      (payload ->> 'latency_ms')::NUMERIC
    )::NUMERIC, 1) AS median_ms,
  ROUND(PERCENTILE_CONT(0.95)
    WITHIN GROUP (ORDER BY
      (payload ->> 'latency_ms')::NUMERIC
    )::NUMERIC, 1) AS p95_ms
FROM (
  SELECT *, timestamp AS request_timestamp,
    jsonb_build_object(
      'method', method, 'endpoint', endpoint,
      'status_code', status_code,
      'latency_ms', response_time_ms,
      'user_agent', user_agent
    ) AS payload
  FROM audit.api_requests
) api
WHERE payload ? 'latency_ms'
```

Answer (2/2)

✅ ANSWER

```
GROUP BY payload ->> 'method'
ORDER BY p95_ms DESC;
```

HW 5: Hourly Error Dashboard from JSONB

CRAZY

TASK: Build an hourly API health dashboard: hour, total requests, GET count, POST count, error count, avg latency. Combine JSONB extraction, FILTER, and DATE_TRUNC.

Answer (1/2)

✅ ANSWER

```
SELECT
  DATE_TRUNC('hour', request_timestamp) AS hour,
  COUNT(*) AS total,
  COUNT(*) FILTER (WHERE
    payload ->> 'method' = 'GET')    AS gets,
  COUNT(*) FILTER (WHERE
    payload ->> 'method' = 'POST')   AS posts,
  COUNT(*) FILTER (WHERE
    (payload ->> 'status_code')::INT >= 400)
                                     AS errors,
  ROUND(AVG(
    (payload ->> 'latency_ms')::NUMERIC
  ), 1)                              AS avg_latency
FROM (
  SELECT *, timestamp AS request_timestamp,
    jsonb_build_object(
      'method', method, 'endpoint', endpoint,
      'status_code', status_code,
      'latency_ms', response_time_ms,
      'user_agent', user_agent
    ) AS payload
  FROM audit.api_requests
```

Answer (2/2)

✅ ANSWER

```
) api
WHERE payload ? 'method'
  AND payload ? 'status_code'
GROUP BY DATE_TRUNC('hour', request_timestamp)
ORDER BY hour;
```

KEY TAKEAWAYS

-> returns JSONB, ->> returns TEXT: Use ->> for display and casting. Use -> for chaining deeper into nested objects.

Always cast ->> results: ->> gives text. Cast to ::INT, ::NUMERIC, ::BOOLEAN, ::TIMESTAMP as needed.

@> for containment filtering: Fast with GIN indexes. Checks if a JSONB value contains a sub-document.

jsonb_array_elements for unnesting: Explodes JSON arrays into rows. Use the _text variant for text arrays.

Flatten JSON into views: Create views that extract and type-cast JSON fields. Analysts query the view without knowing JSON syntax.

What is Next

Tomorrow we learn Views and Materialized Views -- the analyst's delivery format. You will build layered views that clean, enrich, and aggregate data, plus materialized views for expensive queries that need to be fast.